Diffeomorphic Learning for Nonlinear Classification

Revising the Mathematical Flow, Geometry, & Numerical Results

Final Presentation (Version 2)

April 8, 2026

Agenda

1. Motivation and Introduction
2. Mathematical Formulation
3. The Core Geometric Trick: Adding a Dimension
4. Computational Challenges
5. Numerical Tours and Results
6. Implementation & Conclusion

Motivation

Core objective: Learn a smooth global deformation that makes difficult, non-linearly separable classification problems simpler for a final linear model.

Why this matters:

- Many datasets are topologically bound and not linearly separable in their original geometry.
- Deep black-box nonlinear models can be accurate but less interpretable.
- Diffeomorphic learning provides an explicit, invertible geometric transformation governed by physical energy constraints.

Classifier form:

$$x \mapsto F(\phi(x); \theta).$$

Introduction: Geometric Learning View

- ϕ is a **diffeomorphism**: smooth, invertible, and topology-preserving.
- The model moves training samples through a smooth, time-dependent flow (velocity field) to improve separability without tearing the space.
- A simple terminal classifier (e.g. logistic regression) is subsequently applied.

Design principle (The Trade-Off):

- Maximize final prediction accuracy.
- Minimize the complexity (i.e., the physical kinetic energy) of the topological deformation.

Original Cost Function

From Younes (2019), the objective balances deformation effort, model complexity, and data fit:

$$G(\phi, \theta) = \underbrace{D(\mathrm{id}, \phi)^2}_{\text{Deformation Cost}} + \underbrace{\lambda g(\theta)}_{\text{Regularization}} + \underbrace{\frac{1}{\sigma^2}\Gamma\big(F(\cdot;\theta),\, \phi(1)\cdot T_0\big)}_{\text{Terminal Loss}}$$

- $D(\mathrm{id}, \phi)^2 = \inf_v \int_0^1 \|v(t)\|_V^2 \, dt$: In Riemannian geometry, distance is the shortest path (an L^1 length). However, minimizing this L^2 action (which physically mirrors **Kinetic Energy** $\propto \int v^2$) mathematically guarantees the *exact same* shortest path, but forces a smooth, constant-speed flow.
- $g(\theta)$: Standard regularization (e.g., L^2) on terminal classifier parameters θ.
- Γ: Cross-entropy data loss. Yes, the classification loss is evaluated **strictly** and only at the final time $t = 1$, assessing how linearly separable the points $\phi(1) \cdot x$ have become.

Kernels and the Function Space (V)

The velocity field $v(t, x)$ at any time t lives in a **Reproducing Kernel Hilbert Space (RKHS)** defined by a kernel $K(x, y)$.

Choices of Kernels matter theoretically:

- **Gaussian RBF:** Smooth (C^∞) transformations.
- **Matérn Kernels:** Highly valuable theoretically as they are equivalent to Sobolev spaces, allowing control over exact differentiability classes (C^k).
- **Graph-Based Kernels:** Used if the data resides on strict grids or defined graphs (breaking isotropic symmetries).

Affine Transformations: A Theoretical Necessity

The Limitation of pure RKHS:

- RKHS kernels (like Gaussians) vanish at infinity. This means the model struggles to perform simple translations and rigid global rotations.

The Fix (Section 4.3):

- We explicitly construct an extended vector space: $\widehat{v}(t, \cdot) = g(t, \cdot) + v(t, \cdot)$, where g belongs to a Lie algebra (e.g., Affine transforms).
- **Crucial Result:** This breaks the rotational invariance of scalar kernels and massively increases the topological expressivity of the model. *It is not just a speedup, it is a structural necessity.*

How to Optimize? The Kernel Trick & Control Points

Optimizing an infinite-dimensional flow $v(t)$ is impossible numerically.

The RKHS Reduction: Because the end-loss only depends on the trajectories of the N sample points, the Representer theorem guarantees the optimal velocity takes the finite form:

$$v(t, \cdot) = \sum_{k=1}^{N} K(\cdot, z_k(t)) a_k(t)$$

We have transformed an infinite-dimensional problem into finding **finite, time-dependent control vectors** $a_k(t)$ attached to the traveling points $z_k(t)$.

The Complete Reduced Optimal Control Problem

When we include the Affine transformation $g(t,x) = A(t)x + b(t)$, the total velocity field becomes:

$$\widehat{v}(t,\cdot) = g(t,\cdot) + \sum_{k=1}^{N} K(\cdot, z_k(t))\, a_k(t)$$

Plugging this back into our original cost formulation yields our final **computable** objective:

$$\mathcal{E}(a,g,\theta) = \underbrace{\int_0^1 \|g(t)\|_{\mathfrak{a}}^2\, dt}_{\text{Affine Energy}} + \underbrace{\int_0^1 \sum_{k,l} a_k(t)^T K(z_k(t), z_l(t)) a_l(t)\, dt}_{\text{RKHS Energy}} + \lambda g(\theta) + \frac{1}{\sigma^2}\Gamma$$

The updated flow constraints:

$$\dot{z}_k(t) = \widehat{v}(t, z_k(t)), \quad z_k(0) = x_k$$

This completely reduces the continuous setting to an exact ***finite-dimensional optimal control problem*** *over variables a, g, and θ.*

Pontryagin Maximum Principle: Why Applicable Here

We have exactly the PMP setting:

- State equation (ODE) driven by continuous controls $a(t)$.
- Integral running cost + terminal loss evaluated at $t = 1$.
- Smooth dynamics and differentiable objective.

Hence, first-order optimality can be characterized via a **Hamiltonian mechanics** system coupling the forward positions with a backward co-state $p(t)$.

Role in practice: PMP mechanics naturally provide the exact *Backpropagation-Through-Time (BPTT)* equations needed to iteratively extract gradients.

Applying PMP: Continuous Optimality System

Define Hamiltonian (control form):

$$H_a(p, z) = \langle p, f(z, a) \rangle - L(z, a),$$

with the drift defined by the RKHS kernel:

$$f_k(z, a) = \sum_l K(z_k, z_l) a_l.$$

PMP conditions (The physics of the flow):

$$\dot{z} = \partial_p H_a \quad \text{(Data flows forward)}, \qquad \dot{p} = -\partial_z H_a \quad \text{(Error flows backward)}.$$

Boundary condition (Terminal gradient at $t = 1$):

$$p(1) = \frac{1}{\sigma^2} \partial_z \Gamma\big(F(\cdot; \theta), T(z(1))\big).$$

Discretized Final Formula (Used Numerically)

Euler discretization ($t = 0, \ldots, T-1$):

$$z(t+1) = z(t) + \frac{1}{T}\,\partial_p H_a(t).$$

Adjoint recursion (BPTT):

$$p(t-1) = p(t) + \frac{1}{T}\,\partial_z H_a(t), \quad p(T-1) = \frac{1}{\sigma^2}\partial_z \Gamma.$$

Gradient form used in updates (for Adam/CG):

$$u_k(t) = \sum_l K\big(z_k(t), z_l(t)\big)\big(p_l(t) - 2a_l(t)\big).$$

This explicitly establishes mathematically rigorous backpropagation through the geometric deformation!

Overcoming the Topology Barrier (Section 5.1)

The Problem:

- Diffeomorphisms are strictly topology-preserving.
- In 1D, this strictly means **preserving monotonicity** (a line cannot fold over itself). In N-D, it means points maintain their neighborhood structure without tearing or gluing space.
- If Class A (inner circle) is completely surrounded by Class B (outer ring), **no strictly 2D diffeomorphism can separate them** without tearing the space!

The Theoretical Solution (Lifting / The Dummy Dimension):

- According to Younes (2019), adding a dummy dimension is the unique structural solution proposed within this framework to resolve the topological restriction.
- We augment our data with an extra coordinate: $x \to (x, 0)$.
- In this $(d+1)$-dimensional space, the topological barrier is broken. The velocity field simply lifts Class A **over** or **under** Class B in the dummy dimension, flattening them out perfectly for logistic regression.

Scalability and Limits

Method	Typical Complexity	Practical impact
Diffeomorphic Learning	$\mathcal{O}(TdN^2)$ / iter	Main bottleneck: N^2 kernel coupling prevents scaling natively.
Logistic Regression	$\mathcal{O}(NdC)$ / iter	Very fast; strong on near-linear data.
Kernel SVM	$\mathcal{O}(N^3)$	Strong nonlinear baseline but expensive at scale.
MLP	$\mathcal{O}(Ndh)$ / epoch	Scales well linearly with minibatching.

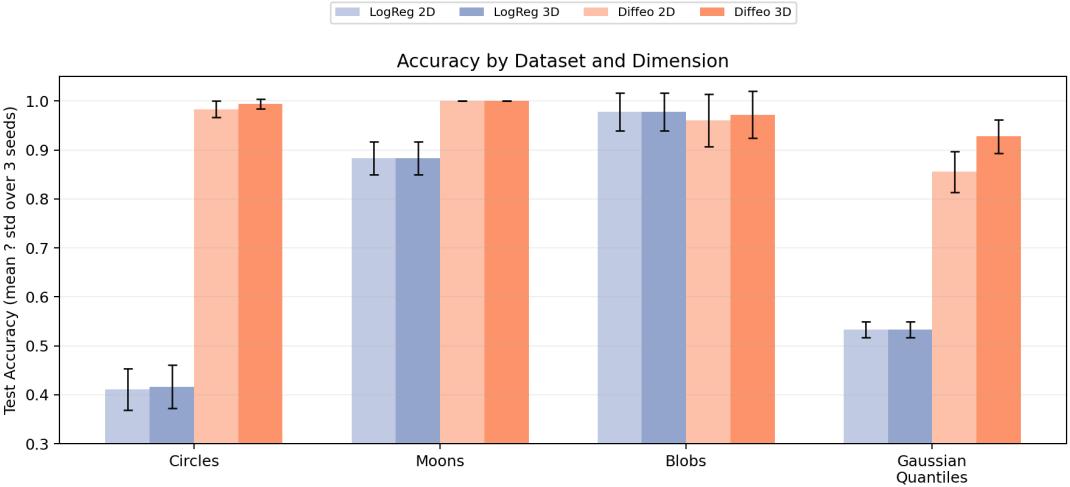
Proposed speedups to bypass N^2 coupling:

- **Control points** ($n \ll N$): Represent flow with sparse landmarks instead of sample points.
- **Mini-batch data term**: Made possible once the control representation is decoupled from N, enabling stochastic gradient descent.
- **Parametric Models (Section 5.2)**: Replace the exact RKHS representation with a neural-net-like parametric form (e.g., $v(t,x) = \sigma(A(t)x + b(t))$), mapped over finite dimensions.

Experimental Setup

- Datasets: circles, moons, blobs, gaussian quantiles.
- 2D baseline vs 3D lifting (The Dummy Dimension approach, injected with Gaussian noise $\sigma = 0.01$).
- Paired seeds: 0, 1, 2.
- Metrics: test accuracy (mean and std).

Performance: 2D vs 3D Across Datasets



Notice the massive accuracy spike on Gaussian quantiles explicitly because of 3D Lifting!

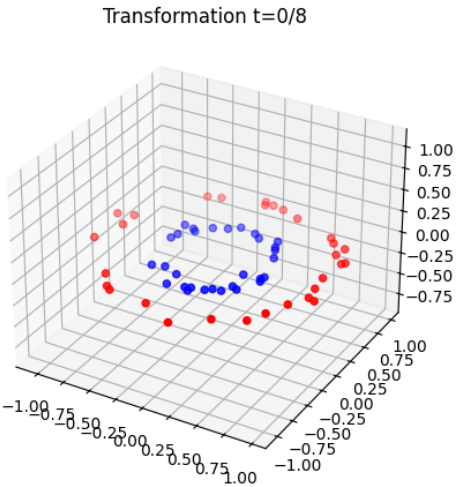
Who Performs Best Where? (Final Accuracy Criterion)

Dataset	LogReg 2D	LogReg 3D	Diffeo 2D	Diffeo 3D	Δ Diffeo
Circles	0.411	0.417	0.983	0.994	**+0.011**
Moons	0.883	0.883	1.000	1.000	+0.000
Blobs	0.978	0.978	0.961	0.972	+0.011
Quantiles	0.533	0.533	0.856	0.928	**+0.072**

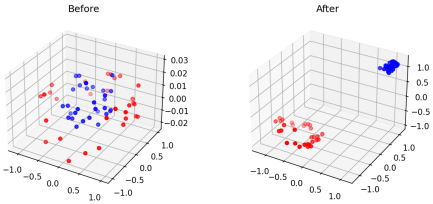
- **Circles**: Strong topological barrier. Diffeo easily handles this via non-linear bending.
- **Gaussian Quantiles**: The Dummy 3D dimension is mathematically necessary to let the algorithm flow inner quantiles over the outer quantiles. Result: a +7.2% leap in performance exactly mimicking theory.

Transformation: Circles Dataset

Evolution

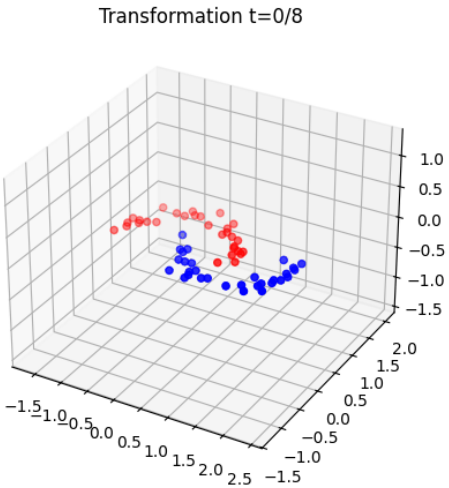


States

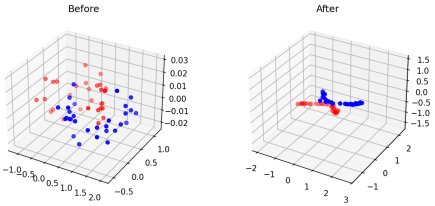


Transformation: Moons Dataset

Evolution

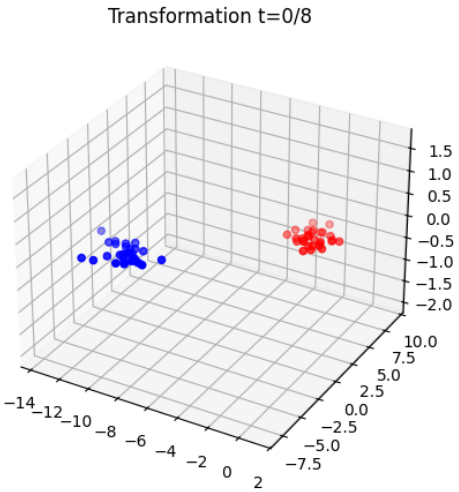


States

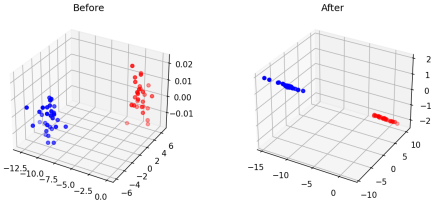


Transformation: Blobs Dataset

Evolution

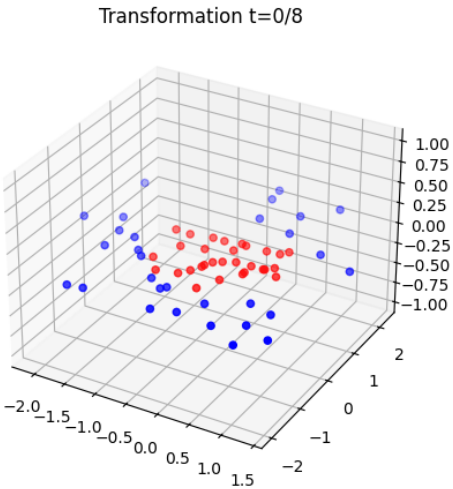


States

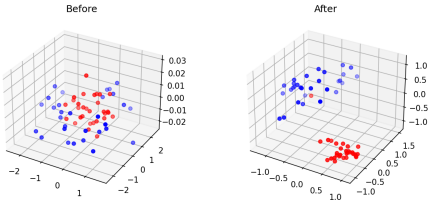


Transformation: Gaussian Quantiles

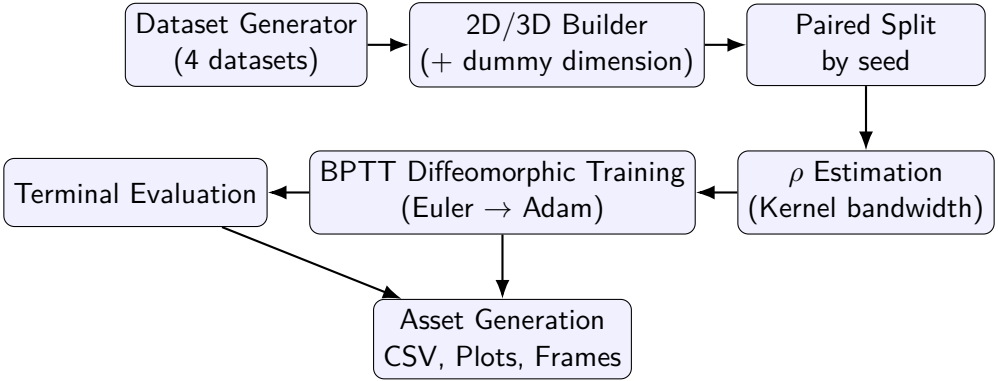
Evolution



States



Implementation Schema



Conclusion

- **Topological Flow:** Diffeomorphic learning is a physically intuitive dynamic geometric model replacing the "black-box" mechanics of standard NNs.
- **The Core Tricks:** Incorporating Affine extensions breaks invariance, while adding a single Dummy coordinate completely solves the restriction of topological preservation.
- **Numerical reality:** The $O(N^2)$ explicit calculation is a heavy bottleneck; future work must concentrate on sparse parametric approximations rather than exact numerical representation.

Thank you.